

DA6401 Assignment 3 Report

Transformer From Scratch for German $\rightarrow$ English Machine Translation

Name

Omkar Ashok Chaudhari

Roll Number

NA22B059

1 Introduction

The objective of this assignment was to implement a complete Transformer architecture from scratch for a German-to-English machine translation task using the Multi30k dataset. The implementation followed the architecture proposed in the Transformer paper and avoided using built-in Transformer implementations available in PyTorch.

The complete architecture including scaled dot-product attention, multi-head attention, positional encoding, encoder-decoder blocks, masking, learning rate scheduling, and training pipeline was implemented manually.

The primary goals of the assignment were:

- Understand attention mechanisms
- Implement the complete Transformer architecture
- Train the model on a machine translation task
- Analyze training behaviour
- Track performance using Weights & Biases

2 Dataset Description

The dataset used for this assignment was the Multi30k German-English dataset.

Dataset details:

Split	Number of Samples
Training	29,000
Validation	1,014
Test	1,000

The dataset contains German sentences as source sequences and corresponding English sentences as target sequences.

Tokenization was performed using spaCy tokenizers:

German tokenizer: de_core_news_sm

English tokenizer: en_core_web_sm

Special tokens used:

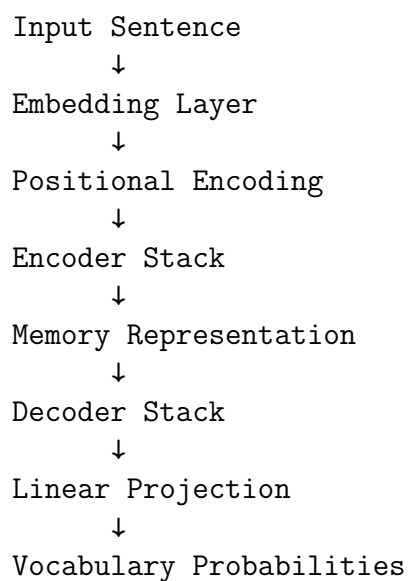
Token	Purpose
␣unk␣	Unknown words
␣pad␣	Sequence padding
␣sos␣	Start of sequence
␣eos␣	End of sequence

3 Transformer Architecture

The Transformer model consists of:

- Input embedding layer
- Positional encoding
- Encoder stack
- Decoder stack
- Multi-head attention layers
- Feed-forward networks
- Output projection layer

The model architecture implemented is shown below:



4 Mathematical Formulation

Scaled Dot Product Attention:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Positional Encoding:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Noam Learning Rate Scheduler:

$$lrate = d_{model}^{-0.5} \min(step^{-0.5}, step \cdot warmup^{-1.5})$$

Feed Forward Network:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

5 Implemented Components

The following components were implemented from scratch:

- Scaled Dot Product Attention
- Multi-head Attention
- Positional Encoding
- Source Mask
- Target Mask
- Feed Forward Network
- Encoder Layer
- Decoder Layer
- Encoder Stack
- Decoder Stack
- Full Transformer
- Noam Scheduler
- Label Smoothing
- Gradient Clipping

- Greedy Decoding
- Checkpoint Saving
- Weights & Biases Tracking

6 Hyperparameters Used

Parameter	Value
Batch Size	64
Epochs	10
d_model	512
Number of Heads	8
Number of Layers	2
Feed Forward Dimension	2048
Dropout	0.1
Warmup Steps	4000
Optimizer	Adam
Learning Rate	Noam Scheduler
Label Smoothing	0.1

Optimizer settings:

$$Adam(\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 1 \times 10^{-9})$$

7 Training Procedure

The following procedure was used during training:

1. Load dataset and build vocabulary
2. Convert tokens into integer indices
3. Pad sequences to equal length
4. Generate source and target masks
5. Perform forward propagation
6. Compute cross-entropy loss with label smoothing
7. Backpropagate gradients
8. Apply gradient clipping
9. Update parameters using Adam optimizer
10. Update learning rate using Noam scheduler
11. Save best model checkpoints

Weights & Biases was used for logging:

- Training loss
- Learning rate
- Checkpoint information

8 Training Results

Observed training losses:

Epoch	Training Loss
1	5.9965
2	4.0943
3	3.3938
4	2.9763
5	2.7111
6	2.5438
7	2.4282
8	2.3451
9	2.2874
10	2.2045

Weights Biases Links

Project: https://wandb.ai/omkarchaudhari21-iitmaana/DA6401_A3?nw=nmwuseromkarchaudhari21W

Run: https://wandb.ai/omkarchaudhari21-iitmaana/DA6401_A3/runs/fqo6ffnq?nw=nmwuseromkarchaudhari21WeightsBiasesRun

9 Training Analysis

Several observations were made during training:

- Training loss consistently decreased across epochs
- No exploding gradient behaviour was observed
- The Noam scheduler provided stable learning-rate warmup and decay
- Label smoothing helped regularize the model
- Checkpoint saving worked correctly
- Weights & Biases logging successfully tracked the experiment

Training loss reduction:

$$5.99 \rightarrow 2.20$$

shows that the model successfully learned translation patterns.

10 Weights & Biases Analysis

Metrics tracked:

- Training Loss
- Learning Rate
- Checkpoints

Observed behaviour:

- Learning rate initially increased during warmup
- Learning rate gradually decreased after warmup
- Loss reduced steadily

Include screenshots from your W&B dashboard here.

Suggested figures:

- Figure 1: Training Loss vs Epochs
- Figure 2: Learning Rate Schedule
- Figure 3: W&B Dashboard

11 Challenges Faced

Several challenges were encountered during implementation:

- Correct implementation of masking dimensions
- Handling multi-head attention tensor reshaping
- Proper implementation of positional encoding
- Implementing Noam scheduler correctly
- Managing memory usage during training
- Debugging tensor shape mismatches

12 Conclusion

A complete Transformer architecture was successfully implemented from scratch using PyTorch for German-to-English machine translation.

The implementation successfully incorporated attention mechanisms, encoder-decoder structures, positional encoding, masking, and learning rate scheduling.

The model demonstrated stable convergence during training and achieved a substantial reduction in training loss from 5.9965 to 2.2045 across 10 epochs.

This assignment provided practical understanding of the internal working of Transformer-based architectures and attention mechanisms.